

Large language models

Large language models

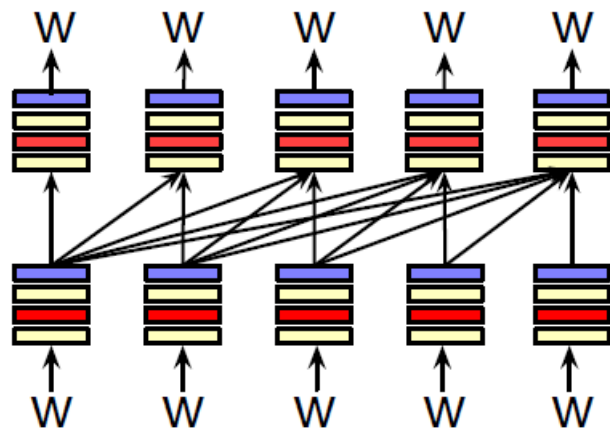
- computational systems that predict the next words
- use large context windows and billions of parameters
- can perform many complex tasks, such as reasoning, coding, translation, creative writing, summarization, or question answering
- usually implemented as large neural networks with a transformer architecture

LLM architectures

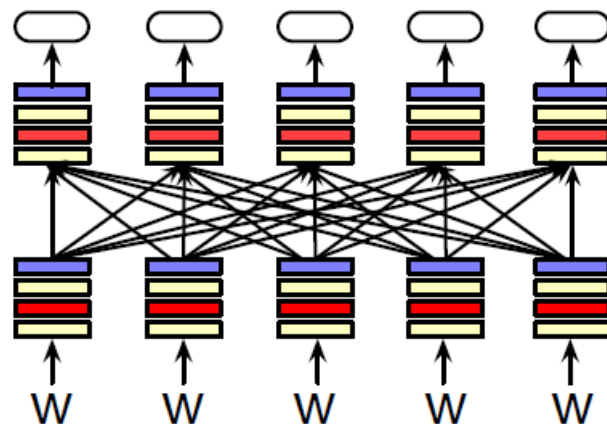
- encoder = “readers”
 - input sequence is transformed into a vector representation of tokens, learn text representations
 - usually uses information from left and right (bidirectional) in the task of masked language modeling
 - often used in tasks like classification, sentiment analysis, NER, etc.
 - BERT, RoBERTa, ALBERT, ...
- encoder-decoder = “translators”
 - generate output based on “understanding” an input sequence
 - used for translation, summarization, etc.
 - T5, BART, Pegasus, ...

LLM architectures

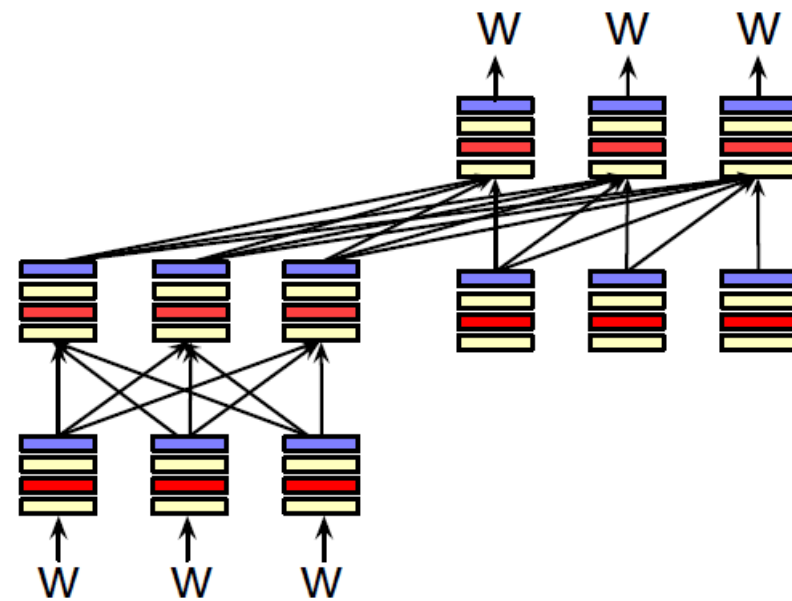
- decoder = “writers”
 - generates tokens based on previous inputs, left-to-right information flow
 - the most prevailing architecture today
 - with enough data and model parameters, no encoder is needed to learn the representation of inputs
 - simpler architecture – no need for two separate parts connected through cross-attention
 - GPT, Llama, Claude, Mistral, Gemini, ...



Decoder



Encoder



Encoder-Decoder

LLM training

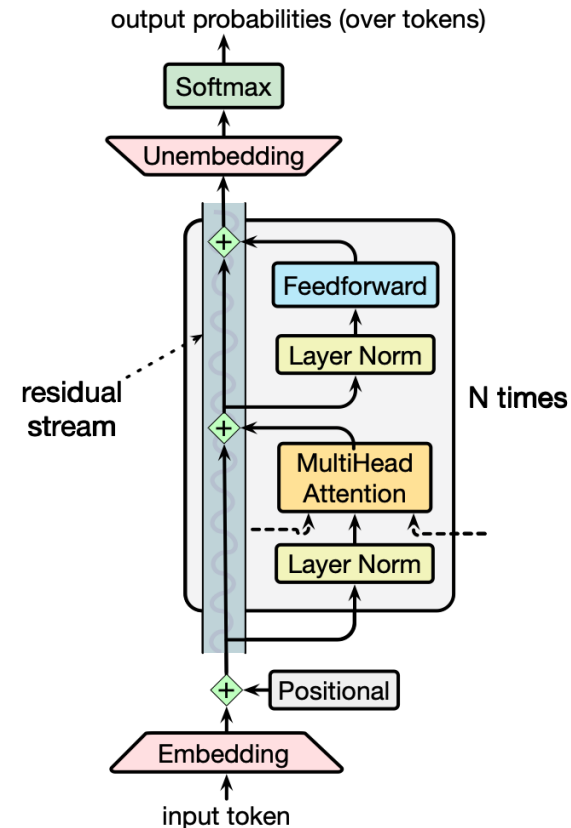
- preprocessing
 - cleaning, tokenization
- pretraining
 - using large corpora in a token prediction task = self-supervised learning
 - cross-entropy loss and backpropagation used to learn model weights
- instruction tuning (finetuning)
 - using a task-specific corpus (e.g., QA, coding, summarization, translation)
 - cross-entropy loss and backpropagation used to update model weights
- alignment
 - final phase, providing accepted and rejected answers
 - reinforcement learning from human feedback or similar learning is applied

LLM evaluation

- perplexity – can be used, but measures only the quality of predicting probabilities of tokens
- other metrics are useful, considering various tasks (e.g., Massive Multitask Language Understanding benchmark) and aspects (toxicity, ethics, energy usage, privacy, bias, etc.)

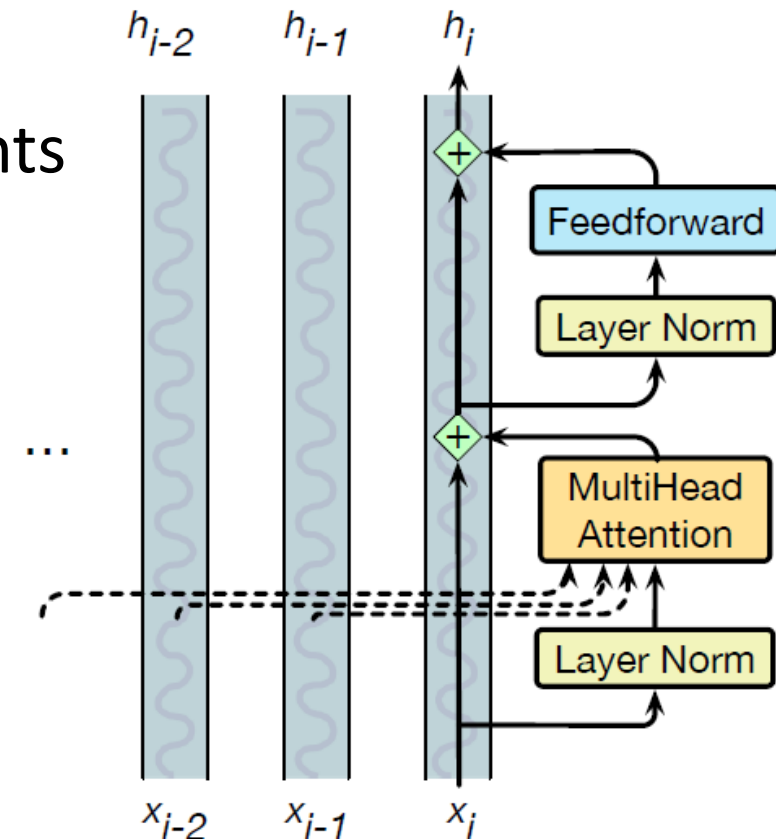
Transformers

- originally (Vaswani et al. 2017. Attention is all you need) had the encoder-decoder architecture as it was used for machine translation
- using self-attention, each token has access to information about surrounding tokens directly (not through the hidden layer vector like in RNNs) – enables creating contextual embeddings
- attention
 - started to be considered the main architectural element, not just a helper to RNNs to handle long sequences, enabling the decoder to access encoder states
 - enabled the encoder and decoder to look at themselves
 - replaced the loops of RNNs, enabling parallel computations

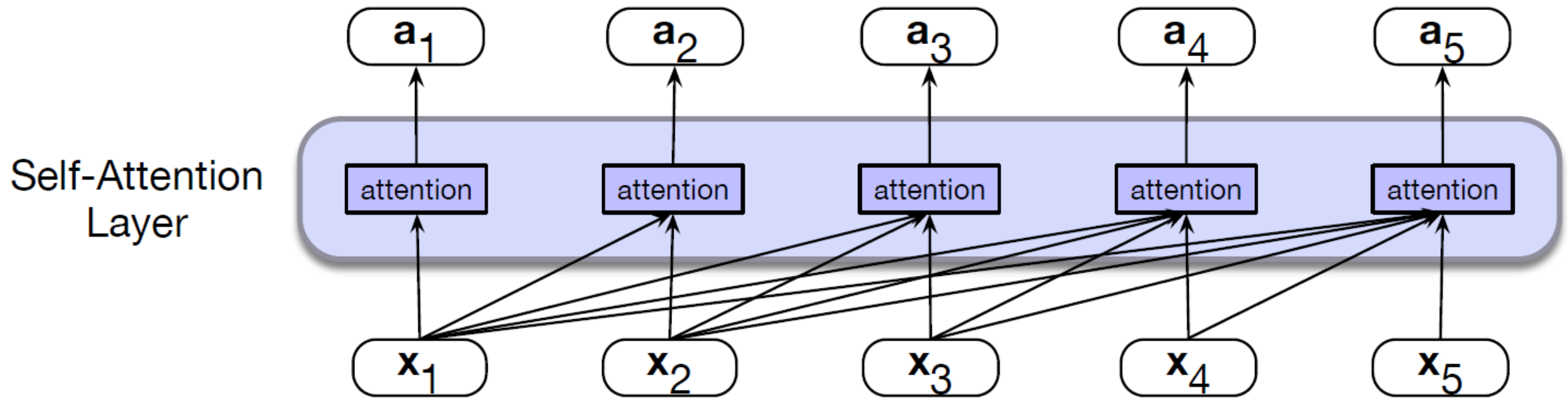


Transformer blocks

- form the layers of transformer models
- tokens are processed in streams, other components read from them and write into them
- the only connection through attention
- contain
 - self-attention
 - feedforward layer
 - residual connections
 - layer normalization



Attention in transformers



Attention in transformers

- simplified version
 - similarity between two vectors is computed (dot product)
 - the similarity is normalized using softmax
 - normalized scores are used for the weighted sum of token embeddings

$$\text{score}_{ij}(x_i, x_j) = x_i \cdot x_j$$

$$\alpha_{ij} = \text{softmax}(\text{score}_{ij})$$

$$a_i = \sum_{j \leq i} \alpha_{ij} x_j$$

- earlier used additive or bilinear forms are replaced by scaled dot-product attention to bring mathematical stability
 - dot product can produce large numbers for high-dimensional vectors
 - it is normalized (divided) by the square root of the number of dimensions to keep the variance=1

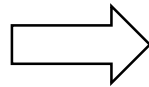
Attention in transformers

- instead of working with raw embeddings (e.g., *bank* can have multiple meanings), they are projected to another space to distinguish the meanings, roles, etc.

$$\text{score}_{ij}(x_i, x_j) = \mathbf{x}_i \cdot \mathbf{x}_j / \text{sqrt}(d_k)$$

$$\alpha_{ij} = \text{softmax}(\text{score}_{ij})$$

$$a_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$



$$\text{score}_{ij}(x_i, x_j) = \mathbf{q}_i \cdot \mathbf{k}_j / \text{sqrt}(d_k)$$

$$\alpha_{ij} = \text{softmax}(\text{score}_{ij})$$

$$a_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Attention in transformers

- the words play different roles (terms borrowed from information retrieval)
 - query = the word for which we search for some information in its surroundings (based on other words)
 - key = word surrounding others, providing its properties
 - value = word, the value of which is being summed
- these representations are obtained using matrices W^Q , W^K , W^V that are learnable

Attention in transformers

- to compute vectors for single words:

$$q_i = x_i W^Q, k_j = x_j W^K, v_i = x_i W^V$$

- vectors x_i are row vectors with some number of dimension d_{model} , e.g., 1×512
- matrices W have dimensions $d_{\text{model}} \times d_k$, e.g., 512×64
- xW has dimensions $1 \times d_k$, e.g., 1×64

Attention in transformers

- to compute all vectors for the entire sequence

$$Q = X W^Q, K = X W^K, V = X W^V$$

- matrices X have dimensions $n \times d_{\text{model}}$ (n ... sequence length), e.g., $n \times 512$
- matrices W have dimensions $d_{\text{model}} \times d_k$, e.g., 512×64
- XW has dimensions $n \times d_k$, e.g., $n \times 64$

Attention in transformers

- with matrices Q, K, and V, the value of attention for all words in the sequence is computed in one step (in parallel)

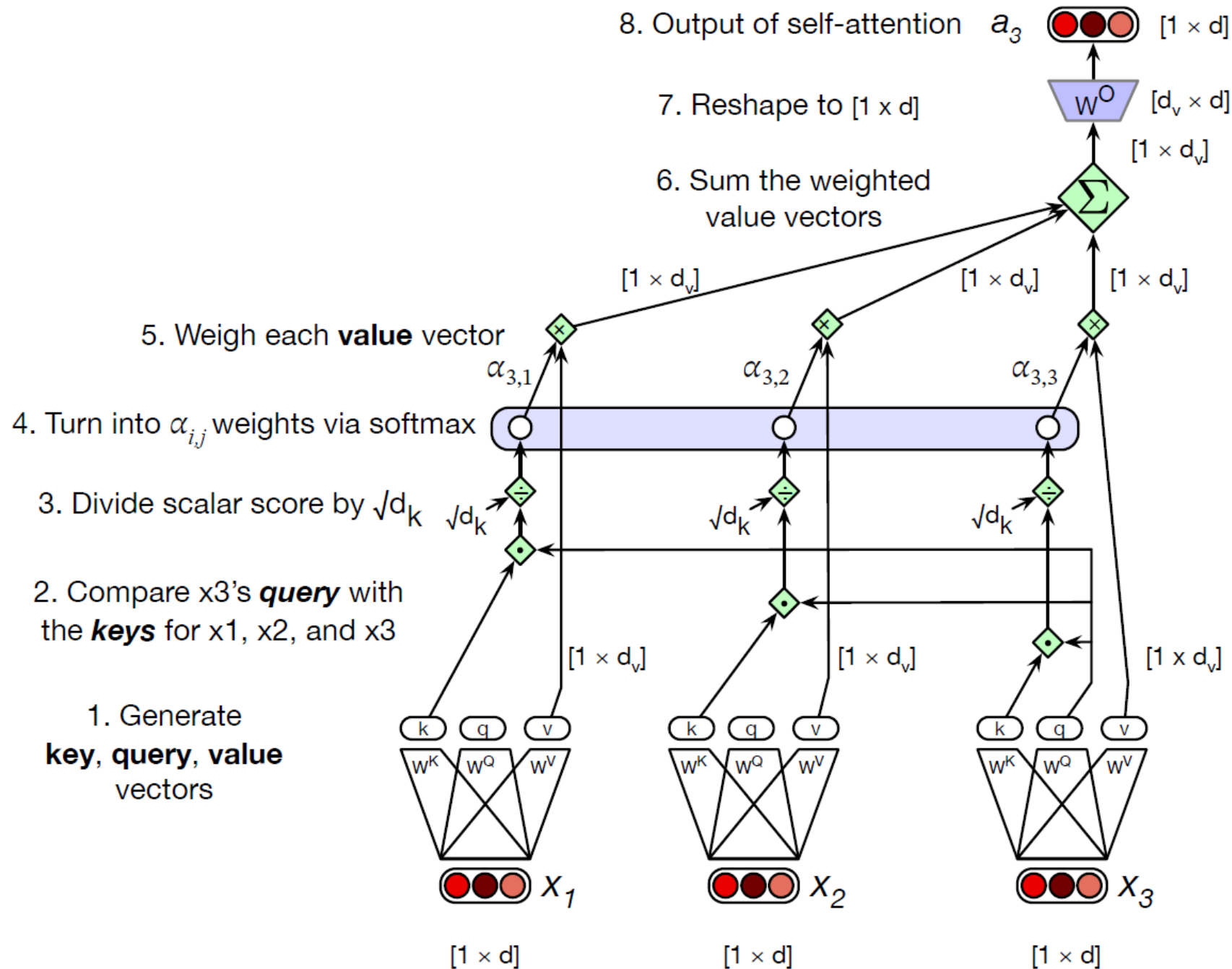
- $Attention(Q, K, V)^* = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$

*also called *attention head*

- QK^T has dimensions $n \times n$
- the final result has dimensions $n \times d_k$

Attention in transformers

- in a transformer, it is necessary to produce a vector (vectors) of the same dimensionality as the input
- it can be achieved by having $d_{\text{model}} = d_k$
 - big matrices = many computations
 - many numbers = some larger values of dot product -> softmax assigns very large values to them and very small to others (i.e., only one token gets attention = “hard” attention) + small gradient
 - one attention tries to capture everything, but only one most important signal is captured
 - alternatively, a smaller vector with the weighted sum of values can be reshaped with another matrix: $a^i = \text{attention}^i W^O$



Attention in transformers

- instead, smaller independent attention heads are used and their result is aggregated
 - more efficient computations (smaller matrices), softmax values better spread among tokens (“soft” attention)
 - each head focuses on one aspect of tokens, the aggregation acts like a committee of experts (ensemble)
 - input with dimensions d_{model} is projected (with W^Q , W^K , W^V) to vectors with dimensions d_k (typically $d_{\text{model}} = d_k * N_{\text{heads}}$, e.g., $512 = 8 * 64$)
 - outputs of each head are concatenated
 - the output matrix W^O is still necessary to combine (eventually reshape to the necessary dimensions) the results of individual attentions:
$$a^i = [\text{head}^1; \text{head}^2; \dots \text{head}^A] W^O$$

Feedforward layer

- process (this is “real” processing; attention adds knowledge about the context) the information gained from attention to update the representation of the token in the stream
- introduce nonlinearity to the model
- two-layer fully connected NN

Feedforward layer

- two-layer fully connected NN

$$\text{FFN}(x_i) = g(x_i W_1 + b_1) W_2 + b_2$$

- g (activation function) usually ReLU or GELU
- W_1
 - expansion matrix (creates larger vectors, e.g., $4 * d_{\text{model}}$)
 - creates “more room” for “thinking” (a problem might be separable in more dimensions, more detailed aspects can be considered, memory capacity is increased)
- W_2
 - contraction matrix (creates vectors with original dimensions, i.e., d_{model})
 - condensing and integrating the learned features to a denser representation, discarding noise from the larger representation
- are the same for all positions in one layer (parameter sharing => parallelization possible), different for each layer

Layer normalization

- normalizing the values of a layer to keep the values on each layer on a comparable scale (similar to input normalization) using the z-score
- the values of a hidden layer, so gradient-based learning is facilitated
- includes two learnable parameters (scale and shift) to eliminate the effect of normalization when needed, shared across positions (vectors of size d_{model} , allowing the model to normalize each dimension independently)

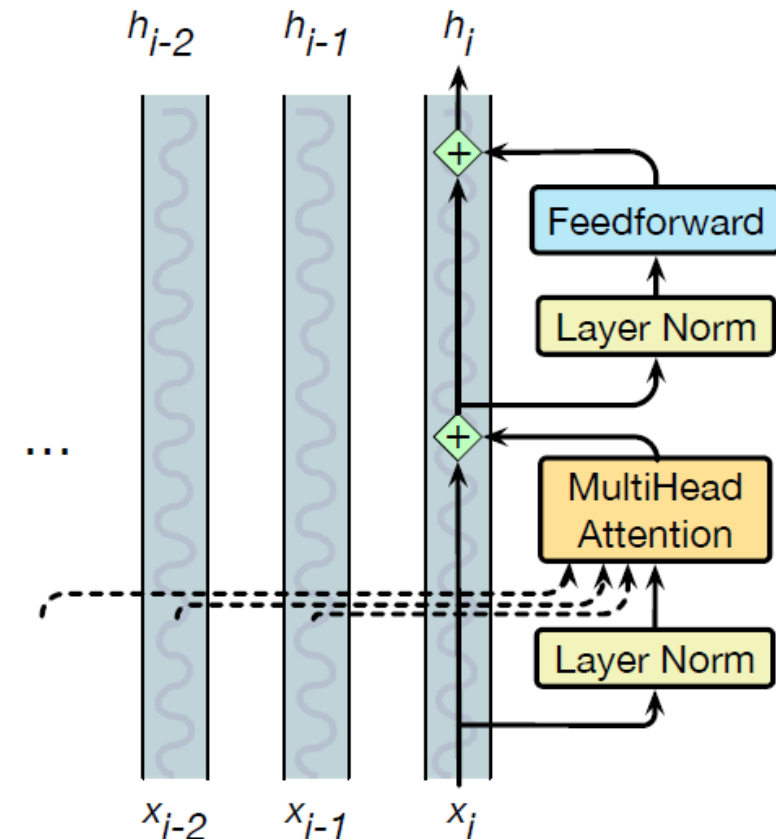
$$\text{LayerNorm}(x) = \gamma \frac{x - \mu}{\sigma} + \beta$$

Layer normalization

- the original transformer had normalization after MHA and FFN
- benefits of having it before:
 - the gradient does not vanish in the normalization layer before reaching MHA or FNN in backpropagation
 - values normalized before "real" learning (MHA, FNN) -> better stability (predictable steps, values in a good range, so not too many neurons die)
 - help handle larger learning rate (not too large inputs)

Transformer block

$$\begin{aligned}t_i^1 &= \text{LayerNorm}(x_i) \\t_i^2 &= \text{MultiHeadAttention}(t_i^1, [t_1^1, \dots, t_N^1]) \\t_i^3 &= t_i^2 + x_i \\t_i^4 &= \text{LayerNorm}(t_i^3) \\t_i^5 &= \text{FNN}(t_i^4) \\h_i &= t_i^5 + t_i^3\end{aligned}$$



Transformer block

- produces a new representation of a position
- only attention accesses information from other streams
- the output has the same dimensionality as the input => blocks can be stacked (from 12 layers in small models to around 100 in very large models)

Transformer block

- computations (attention, FNN) on each layer are parallelized (matrix multiplications), all inputs are stored in matrix X:

$$O = X + \text{MultiHeadAttention}(\text{LayerNorm}(X))$$

$$H = O + \text{FFN}(\text{LayerNorm}(O))$$

Masking

- sometimes, we don't want some positions to participate in computations
- a padding mask sets $-\infty$ to the appropriate positions (softmax will become almost 0), so no attention is paid to them
- is achieved with the addition of a square matrix M ($-\infty$ for positions to be masked, 0 in other positions):

$$Attention(Q, K, V)^* = softmax\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

Masking

- padding mask
 - all sequences are represented with the same size, are padded with zeros if they are shorter
 - a padding mask says no attention is paid to the zeros
 - has $-\infty$ in some columns

Masking

- look-ahead mask
 - are used by decoders to hide future positions (looking at future tokens is not desired in a token-prediction task – the decoder would know the correct answer)
 - has $-\infty$ above the main diagonal

N

q1•k1	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4

N

Inputs

- the input embedding x is actually the sum of a token embedding and a positional embedding (adds some positional signal)
- otherwise, the tokens will be treated as a bag of words (all tokens are processed at the same time in parallel)
- original tokens are represented by one-hot vectors and used to extract their embeddings from an embedding matrix

The diagram illustrates the process of extracting embeddings from an embedding matrix. On the left, a matrix of size $N \times |V|$ is shown, where N is the number of tokens and $|V|$ is the vocabulary size. This matrix contains one-hot vectors, represented as rows of 0s and 1s. The first three rows are explicitly shown, with ellipses indicating more rows. The first row has a 1 at the 4th position, the second row has a 1 at the $|V|$ th position, and the third row has a 1 at the 1st position. This matrix is multiplied (indicated by $\times$) by an embedding matrix E of size $|V| \times d$, where d is the embedding dimension. The embedding matrix E is represented by a light blue vertical rectangle. The result of the multiplication is an output matrix of size $N \times d$, represented by a light green rectangle. The equation is summarized as: $N \times |V| \times |V| \times d = N \times d$.

Absolute positions

- order in the sequence
 - adding 1 to the first token, 2 to the second, etc.
 - most straightforward, but not bounded, high order outweighs embeddings of tokens
- learned positions (e.g., BERT)
 - each position has its own embedding learned during training
 - earlier positions appear more than later -> later positions are trained poorly

Absolute positions

- fixed sinusoidal (original transformer)

- every position is given a vector based on the functions sin and cos

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

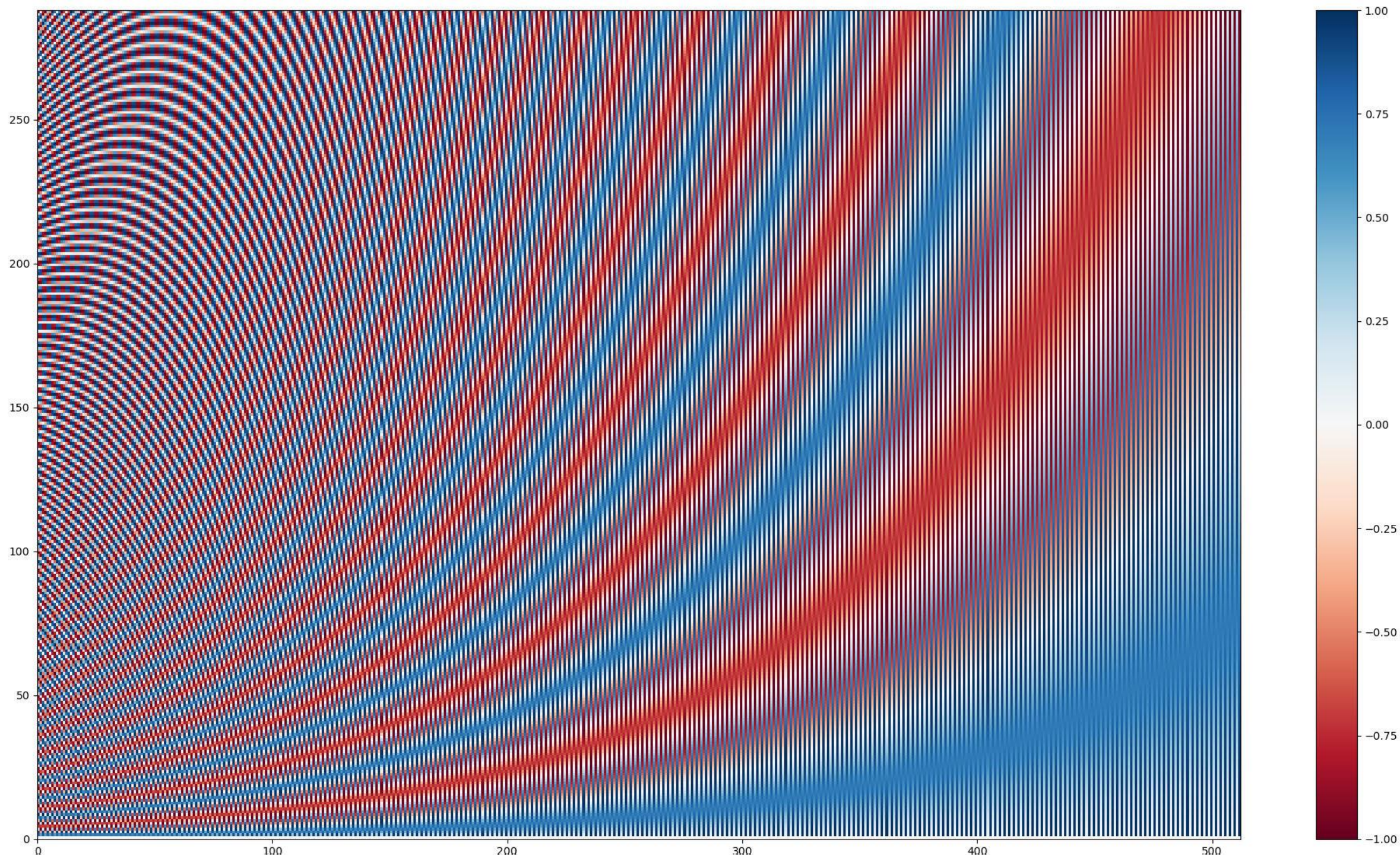
pos ... position of token (0, 1, 2, ...), i ... index in the position vector

- understanding

- every pair of values in a positional embedding represents “one coordinate” of a vector moving on a unit circle
 - every pair “moves at a different speed” (first point slower)
 - a constant in the formula ensures that the slowest movement lasts longer than we have possible positions
 - all positions together are unique

- benefits

- values bound between -1 and 1
 - the embeddings for any position can be computed
 - the relative distance of positions remains the same no matter where it occurs (e.g., between positions 2 and 5, and between 102 and 105) – the dot product in QK^T basically computes the angle between two vectors (a positional component was added)



Relative positions

- instead of using an absolute position (e.g., “the information you look at is at position 5”), the model works with the relative positions (e.g., “the information is three positions to the left”)
- instead of adding a positional embedding to the input, a bias is added to the score for i th and j th token

$$\frac{QK^T}{\sqrt{d_k}} + B_{i-j}$$

- T5: the bias can be learnable (better for shorter dependencies, can fine-tune its focus), but not for every position – instead, bucketing can be used (some bias for distances 1 to 4, 5 to 9, 10 to 20, ...)
- BLOOM: the bias can be fixed (i.e., zero parameters), growing with a distance (suitable also for long distances); in ALiBi (Attention with linear biases), $B = -m * |i-j|$ where $|i-j|$ is the distance, m is the fixed slope for each attention head (e.g., $1/2, 1/4, 1/8, \dots, 1/256$ for 8 heads – heads with steeper slopes, e.g., $1/2$, are nearsighted, heads with more gentle slopes, e.g., $1/256$ are more farsighted)

Residual stream output

- at the end of a stream, a representation of the input updated with multiple attention and FFN layers is provided
- this output represents a contextual embedding
- has the same dimensionality as the input
- the new representation (often pooled output embeddings) can be used, e.g., for semantic search and similarity calculation, or can serve as input to other models
- the output can be further processed by “heads”

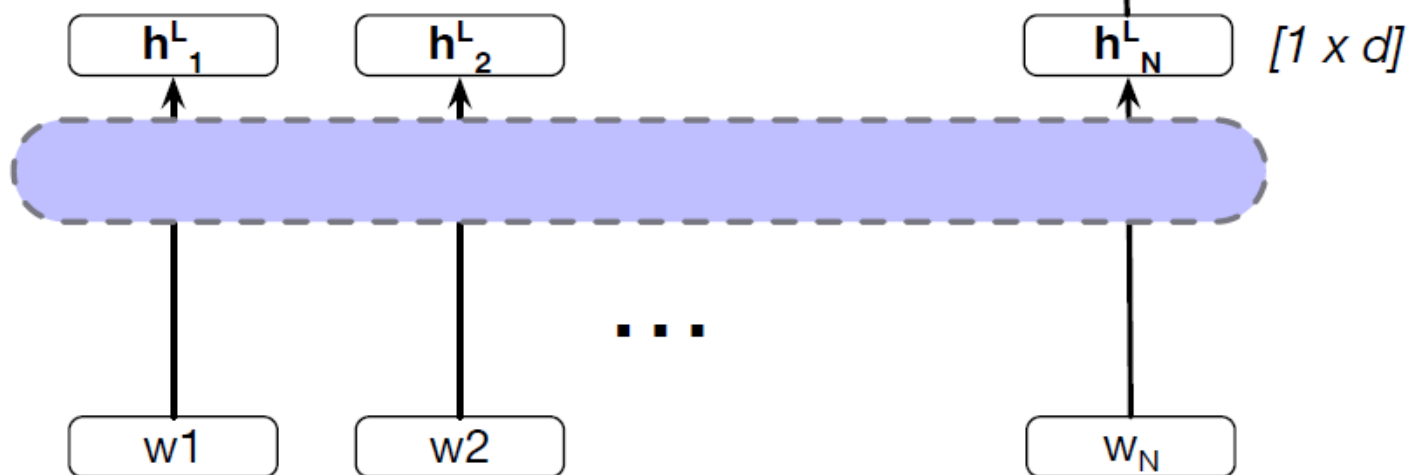
Language modeling head

- enables the output of the last layer of a ***decoder*** to be transformed into a probability distribution over tokens
- consists of
 - linear layer (weights shared across all streams) transforming the transformer output to the vocabulary size (often uses transposed embedding matrix = weight tying)
 - softmax layer

Language Model Head

takes h_N^L and outputs a distribution over vocabulary V

Layer L
Transformer
Block



Word probabilities $[1 \times |V|]$

Softmax over vocabulary V

Logits $[1 \times |V|]$

Unembedding layer $[d \times |V|]$

Generation of tokens in decoders

- a next token is sampled from a probability distribution provided by the decoder
 - greedy decoding = looks at the token with the highest probability
 - random sampling
 - a token is chosen randomly considering the probability distribution
 - this probability distribution can be “modified” (temperature, top-k, top-p) as rare words constitute a large portion of the distribution (=> weird sentences)

Generation of tokens in decoders

- greedy decoding
 - choosing the most likely token, although others are almost as likely as the first
 - is predictable, deterministic, fast
 - good for applications where a correct answer is good (e.g., math)
 - some diversity in generation might be beneficial
- temperature sampling
 - the raw scores (logits) are divided by a number T before softmax
 - $T < 0 \Rightarrow$ high scores become higher, low scores lower
 - $T > 0 \Rightarrow$ scores become more similar, rare tokens now have a higher chance to be picked
 - T almost 0 $\Rightarrow$ deterministic, the most likely token has almost 100% probability to be chosen

Generation of tokens in decoders

- top-k
 - probability of 0 is set to all tokens, except the k most likely tokens
 - prevents very rare words from being picked
- top-p (nucleus)
 - keeps only the most likely tokens that cumulatively produce the probability p
 - the number of tokens to choose from changes dynamically

Decoded training

- the task is to predict the next token
- instead of providing the generated token, correct tokens are provided (teacher forcing) – enables all calculations to be done in parallel
- look-ahead (causal) mask is used in self-attention, so the model does not see the tokens to be predicted
- during training, the output of each stream is used to compute the loss for each token (which is then averaged over all tokens in the sequence)

Decoded inference

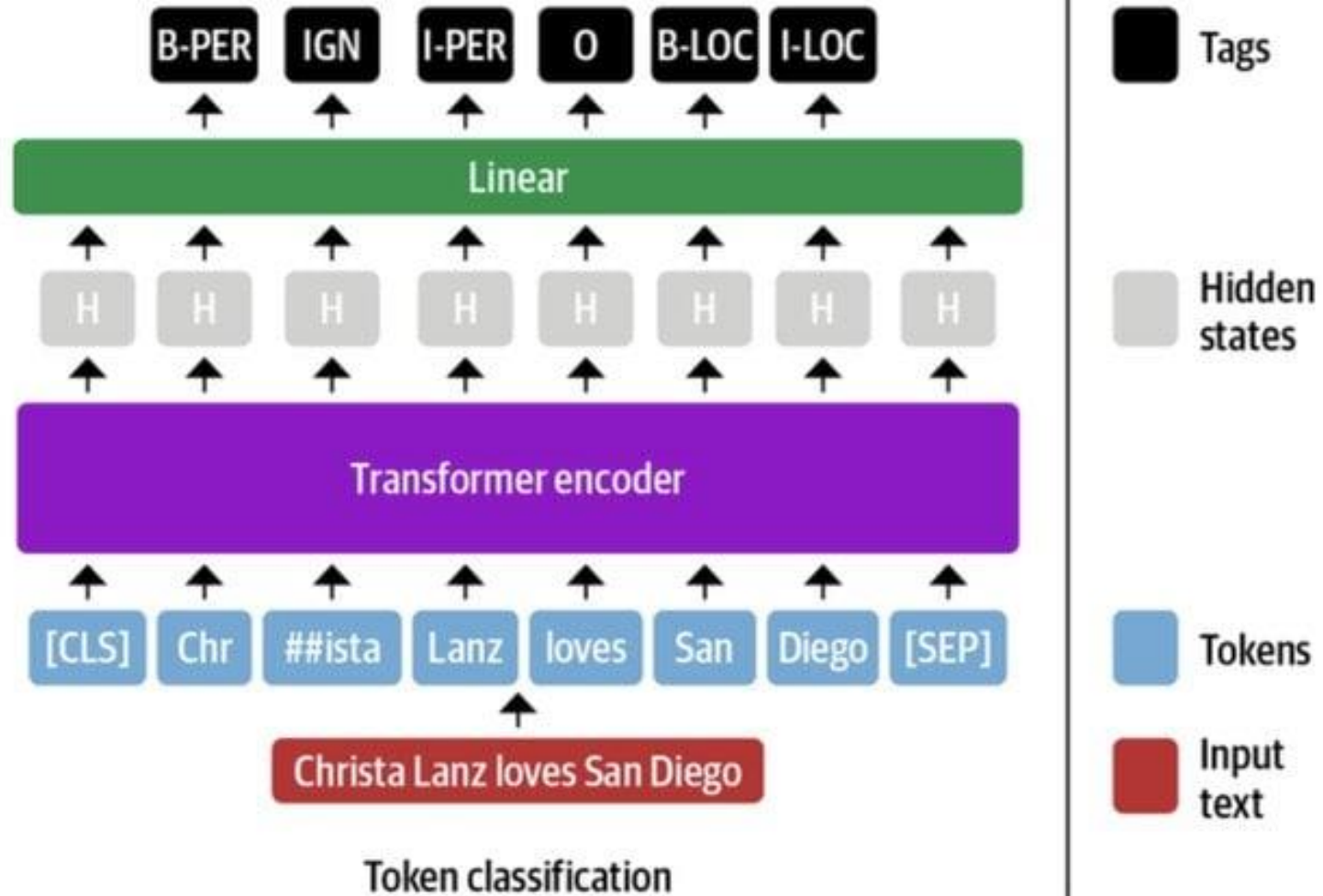
- no “teacher” is available, the model works in the autoregressive mode
 - the output from the last stream used to sample a token, which is appended to the input sequence, which is again processed by the decoder
 - cannot be done in parallel, so it relies on a loop
 - caching the values of K and V matrices for previous inputs is used to speed up computations, only Q for the new token is computed

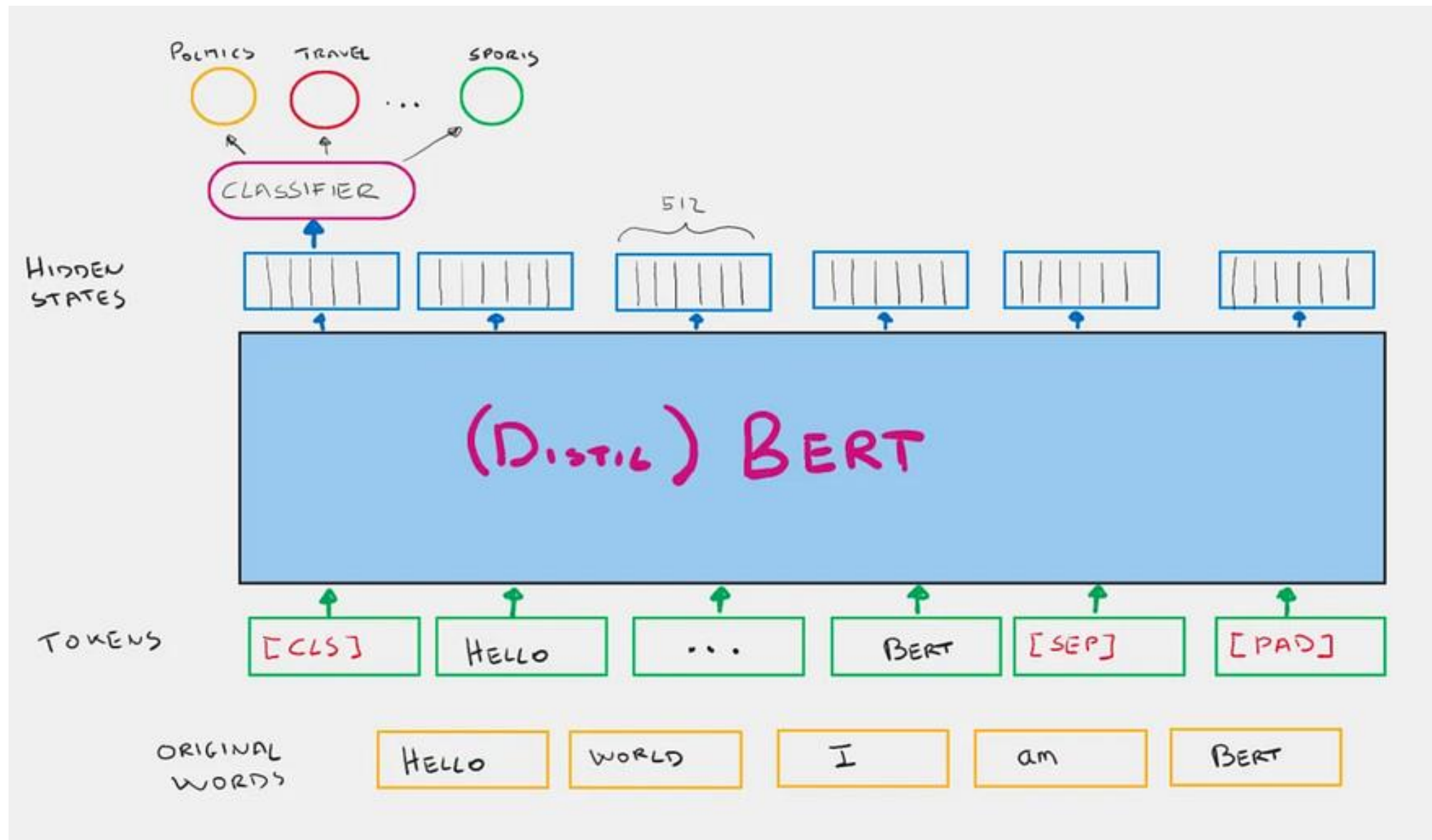
Decoded inference

- the output is often generated based on an input piece of text (prompt)
 - this serves as a basis for autoregressive generation
 - the entire prompt can be processed (=prefill) in parallel (similarly to training), and the output of the last stream is used to sample the next token
- labeled examples in the prompt = few-shot learning
 - can improve performance (this learning does not update the weights of the model, is called in-context learning)
 - through self-attention, the model identifies the repeating patterns (e.g., input:label)

Encoder head

- the task of encoders (readers) is to learn representations
- the last layer can be followed by a task-specific head, e.g.,
 - classification head for each token in NER
 - sequence classification head using pooled information
 - typically the CLS (classification) token is artificially added to the beginning of each sequence, which represents the summary of the entire input
 - mean of max pooling can be used, too
- there can be multiple heads, so multiple tasks can be learned
- auxiliary heads (any of the heads above and others) can be at the top or in the middle of an encoder to facilitate learning (e.g., to add some error signal to prevent vanishing gradient, to produce correct answers earlier = more efficient learning); they are not used in inference





Encoder training

- masked language modeling (MLM)
 - a typical approach to encoder training
 - a portion of tokens (e.g., 15%) is selected for processing and the loss is computed only for them (not for the other as it is easy to predict a token when a model knows it)
 - from the selected tokens
 - 80% are hidden (replaced with a special [MASK] token) – the model learns from surrounding tokens to deduce missing information
 - 10% are replaced by random words – learns the model not to trust everything and to detect errors
 - 10% is kept original – learns the model to trust some inputs, ensures the embeddings for real words are of high quality and stable (in inference, nothing is masked)

Encoder training

- next sentence prediction
 - an auxiliary task (to MLM) in the original BERT
 - two sentences A and B are provided; B either follows A in a real text or not; a classification head using the [CLS] token should decide this correctly
 - sometimes considered too easy as A and B can have different topics (vocabularies)
- sentence order prediction
 - similar to next sentence prediction
 - there are more sentences and the model must decide whether they are in a correct order or swapped (the sentences really appear together, but possibly in a different order)
 - the number of sentences can be different (ALBERT – 2, PEGASUS – approx. 10-20, BigBird – approx. 50-100+, etc.)

Encoder training

- ELECTRA*-style training (replaced token detection)
 - one encoder (generator) is trained only to fill in masked tokens
 - another encoder (discriminator)
 - decides whether a token is original (in the source text) or replaced – binary cross-entropy at the token level is used as the loss function (instead of multiclass cross-entropy in BERT)
 - uses all tokens (not only 15% like traditional MLM in BERT) – much less computation is needed to achieve the same results as BERT
 - eliminated the [MASK] token mismatch – never sees the [MASK] token (BERT sees this token in training but not in inference)

* Efficiently Learning an Encoder that Classified Token Replacements Accurately

Encoder-decoder connection

- encoder produces contextual representations of inputs
- decoder generates outputs based on previously generated outputs and encoder states
- they are connected through cross-attention – similar to self-attention, but the vectors come from different places
 - queries – from decoder's previous layer (what to look at next)
 - keys and values – encoder's final output (what is available on input and how important it is)

